Objets comme structures de données

En JavaScript, les objets sont des structures de données qui permettent de regrouper des valeurs et des fonctions sous une même entité. Ils sont particulièrement utiles pour organiser et manipuler des données complexes. Cela évite d'avoir à gérer de nombreuses variables indépendantes, ce qui peut devenir difficile à suivre.

Par exemple, au lieu de créer plusieurs variables pour stocker les propriétés d'une voiture, on peut créer un objet `personnage` qui regroupe toutes ces informations.

```
let personnage = {
    nom: "Héros",
    positionX: 100,
    positionY: 150,
    vitesse: 10,
    vie: 100,
    estMort: false,
    image: new Image(),
    imageSrc: "images/hero.png",
};
```

Attention : Dans l'exemple ci-dessus, la propriété `image` est initialisée avec `new Image()`, mais l'attribut `src` de cette image n'est pas encore défini. Il faut le faire séparément pour charger l'image correctement.

```
personnage.image.src = personnage.imageSrc;
```

Syntaxe de création d'un objet

Un objet est créé en utilisant des accolades `{}`. À l'intérieur des accolades, on définit des paires clé-valeur séparées par des deux-points `:`. Chaque paire est séparée par une virgule. La clé (ou propriété) est un nom qui identifie la valeur associée. La valeur peut être de n'importe quel type de données, y compris des nombres, des chaînes de caractères, des tableaux, ou même d'autres objets.

Accéder aux propriétés d'un objet

On peut accéder aux propriétés d'un objet en utilisant la notation par point ou la notation par crochets. La notation par point est la plus courante et la plus lisible. Cependant, la notation par crochets est utile lorsque le nom de la propriété contient des espaces ou des caractères spéciaux, ou lorsqu'il est stocké dans une variable.

On accède aux propriétés de l'objet `personnage` comme suit :

```
oContexte.fillText(personnage.nom, personnage.positionX, personnage.positionY);
oContexte.fillText("Vie : " + personnage["vie"], 10, 20);
```

Modifier les propriétés d'un objet

Comme avec les variables, on peut modifier la valeur des propriétés d'un objet en utilisant la notation par point ou par crochets et en utilisant l'opérateur d'affectation `=`.

```
personnage.nom = "Nouveau Héros"; // Modifie le nom du personnage
personnage["vie"] -= 10; // Réduit la vie du personnage de 10
personnage["positionX"] += 20; // Déplace le personnage de 20 unités vers la
droite
```

Ajouter ou supprimer des propriétés

On peut ajouter de nouvelles propriétés à un objet en utilisant la notation par point ou par crochets. Si la propriété n'existe pas, elle sera créée.

```
personnage.age = 30;
personnage["metier"] = "Guerrier";
```

Pour supprimer une propriété, on utilise l'opérateur `delete`.

```
delete personnage.vie;
```

Pourquoi utiliser des objets ? Comparaison pratique

Sans objets (compliqué à gérer) :

```
// Variables séparées - difficile à organiser
let heroNom = "Héros";
let heroX = 100;
let heroY = 150;
let heroVie = 100;

let ennemiNom = "Gobelin";
let ennemiX = 300;
let ennemiY = 200;
let ennemiVie = 50;

// Difficile de passer toutes ces variables à une fonction
function deplacerPersonnage(x, y, vitesse, nom) {
    /* ... */
}
```

Avec objets (organisé et clair) :

```
// Tout regroupé logiquement
let hero = {
    nom: "Héros",
    x: 100,
    y: 150,
    vie: 100,
};

let ennemi = {
    nom: "Gobelin",
    x: 300,
    y: 200,
    vie: 50,
};

// Facile de passer un objet complet
function deplacerPersonnage(personnage, vitesse) {
    personnage.x += vitesse;
}
```

Utilisation pratique dans un jeu Canvas

Voici comment utiliser un objet personnage dans une boucle de jeu réelle :

```
// Créer le personnage
let hero = {
    x: 100,
    y: 150,
    largeur: 32,
    hauteur: 32,
    vie: 100,
    vitesse: 5,
    couleur: "blue",
};

// Fonction de mise à jour (appelée 60 fois par seconde)
function mettreAJourJeu() {
    // Déplacer le héros avec les flèches du clavier
    if (touches.ArrowLeft) {
        hero.x -= hero.vitesse;
    }
    if (touches.ArrowRight) {
        hero.x += hero.vitesse;
    }

    // Empêcher de sortir de l'écran
    if (hero.x < 0) hero.x = 0;
    if (hero.x > canvas.width - hero.largeur) {
        hero.x = canvas.width - hero.largeur;
    }
```

```
    // Dessiner le héros
    contexte.fillStyle = hero.couleur;
    contexte.fillRect(hero.x, hero.y, hero.largeur, hero.hauteur);

    // Afficher les informations
    contexte.fillStyle = "white";
    contexte.fillText("Vie: " + hero.vie, 10, 30);
    contexte.fillText("Position: " + hero.x + ", " + hero.y, 10, 50);
}
```

Créer plusieurs objets similaires

L'avantage des objets devient évident quand on gère plusieurs entités :

```
// Créer plusieurs ennemis facilement
let ennemi1 = { nom: "Gobelin", x: 300, y: 100, vie: 30, couleur: "green" };
let ennemi2 = { nom: "Orc", x: 400, y: 150, vie: 50, couleur: "red" };
let ennemi3 = { nom: "Troll", x: 500, y: 200, vie: 80, couleur: "brown" };

// Les mettre dans un tableau pour les gérer ensemble
let ennemis = [ennemi1, ennemi2, ennemi3];

// Mettre à jour tous les ennemis d'un coup
ennemis.forEach(function (ennemi) {
    ennemi.x -= 2; // Tous bougent vers la gauche

    // Dessiner chaque ennemi
    contexte.fillStyle = ennemi.couleur;
    contexte.fillRect(ennemi.x, ennemi.y, 25, 25);
});
```